CSE 4345: Big Data Analytics

Week 5, Part 2 — Seminal Research & Case Study

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Part 2 Overview — Week 5

Today's Agenda

1. Research Paper — Vavilapalli et al. (2013): YARN
2. Case Study — LinkedIn YARN Cluster (2013–2016)
3. Live Exercise

Seminal Paper Covered

1. Vavilapalli, V. K., et al. (2013). *Apache Hadoop YARN: Yet Another Resource Negotiator.* ACM SoCC 2013. Santa Clara, CA.

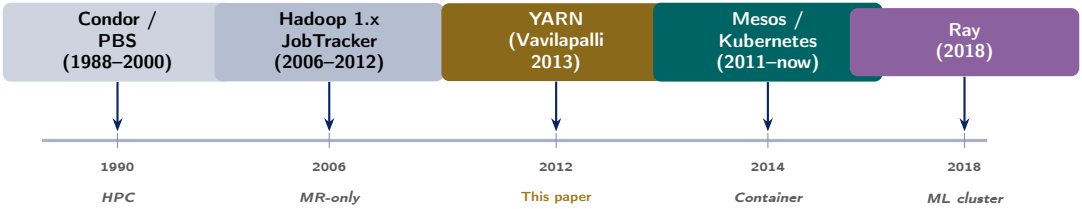
Case Study

LinkedIn YARN Cluster (2013–2016): running MapReduce, Apache Spark, Apache Samza, and Apache Tez on a **single 5,000-node cluster**, processing 500 TB/day for professional-network analytics.

CLO Addressed

Research Paper — Vavilapalli et al. (2013): YARN

Research Landscape: Cluster Resource Management



The Core Problem YARN Solved

Hadoop 1.x's **JobTracker** mixed resource management and MapReduce-specific scheduling into a single process. This prevented any other framework (Spark, Tez, Samza) from running on the same cluster, and hit a hard scalability ceiling of ≈4,000 nodes. YARN separated these concerns entirely.

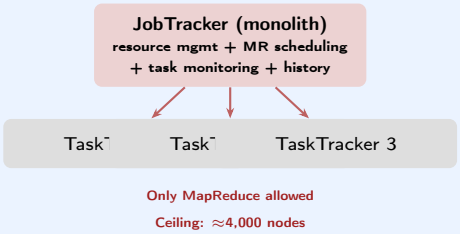
Vavilapalli et al. (2013): Publication Context

Full Citation

Vavilapalli, V. K., Murthy, A. C., Douglas, C., Agarwal, S., Konar, M., Evans, R., Graves, T., Lowe, J., Shah, H., Seth, S., Saha, B., Curino, C., O'Malley, O., Radia, S., Reed, B., & Baldeschwieler, E. (2013). **Apache Hadoop YARN: Yet Another Resource Negotiator.** In *Proc. 4th ACM Symposium on Cloud Computing (SoCC '13)*. Santa Clara, CA.

Venue: ACM SoCC — top cloud systems venue
Cited by: >3,000 (Google Scholar, 2024)
Authors: 16 engineers from Yahoo!, Hortonworks, and Microsoft — all active Hadoop committers

The Hadoop 1.x Bottleneck



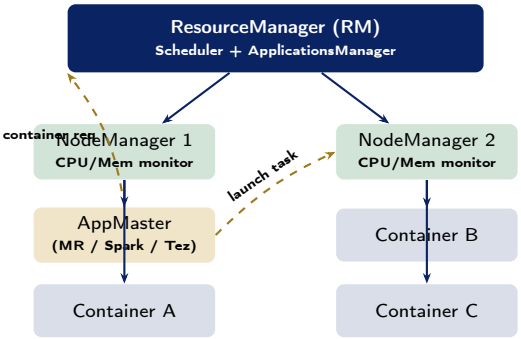
Context: Why Now?

By 2012, Yahoo! ran 40,000-node clusters but a *single*

Technical Contribution 1 — YARN Three-Layer Architecture

The Three Layers

1. **ResourceManager (RM)** — one per cluster
   - **Scheduler**: decides which application gets which containers; pluggable (FIFO, Capacity, Fair)
   - **ApplicationsManager**: accepts job submissions, negotiates a container for each ApplicationMaster
2. **NodeManager (NM)** — one per node
   - Launches and monitors containers
   - Reports CPU + memory usage to RM
   - Manages local log aggregation
3. **ApplicationMaster (AM)** — one per



Technical Contribution 2 — Container Model & Schedulers

Container: The Resource Unit

A **Container** is a bundle of resources on a specific node:

- `{vcores: N, memory: MMB}`
- The AM requests containers with specific resource requirements; the Scheduler decides which node to assign each container to
- Containers are **isolated** via Linux cgroups (CPU) and memory limits enforced by the NM
- If a container exceeds its memory limit, the NM **kills it immediately** to protect other containers

YARN Schedulers (Pluggable)

Scheduler	Policy
FIFO	One queue; first-come first-served. Simple but starves long jobs.
Capacity	Multiple queues with capacity guarantees; excess capacity shared. Used by Yahoo!, LinkedIn.
Fair	Jobs share resources equally; small jobs get fast turnaround. Used by Facebook, Twitter.

Multi-Framework Enablement

Because the AM is *per-application* and handles

Technical Contribution 3 — Fault Tolerance & Timeline Server

Layered Fault Tolerance in YARN

YARN delegates fault handling to three layers:

1. **NodeManager failure**: RM detects missing heartbeat (>10 min) and marks all containers on that node as failed. The AM is notified and re-requests containers elsewhere.
2. **ApplicationMaster failure**: RM restarts the AM (up to a configured max attempts, default 2). The new AM can recover job state from HDFS checkpoints if the framework supports it (MapReduce: yes; early Spark: no).
3. **ResourceManager failure**: YARN 2.4+ added **RM High Availability** via ZooKeeper — an Active RM and a Standby RM; automatic failover in <60 seconds.

Timeline Server: Observability

The **Application Timeline Server** (ATS) is a YARN component introduced alongside YARN:

- Stores per-application and per-container event history
- Accessible via REST API after job completion
- Powers the **YARN ResourceManager UI** (http://rm-host:8088)
- Enables post-mortem debugging: why did container C0001 fail? when did each task start?

Hadoop 1 vs. YARN Comparison

Impact Today — Vavilapalli et al. (2013)

YARN's Lasting Influence

- **Apache Spark on YARN** (2013+): Spark's default cluster mode in enterprise Hadoop; YARN manages executor containers while Spark's AM handles DAG scheduling
- **Apache Tez** (2013): DAG-based successor to MapReduce; Hive and Pig rewrote execution engines to use Tez on YARN, reducing latency by 5–10×
- **Apache Samza** (LinkedIn, 2013): stream processing framework designed natively for YARN containers
- **YARN vs. Kubernetes** (2018–present): Kubernetes has largely replaced YARN for new deployments; but the *three-tier design* (global scheduler / node agent / per-app master) is

CMU 15-721 / Harvard CS265 Perspective

- CMU 15-721 frames YARN's Scheduler as a **multi-resource bin-packing problem**: fitting {vCPU, RAM} containers into heterogeneous nodes is NP-hard; YARN uses *dominant resource fairness* (DRF)
- Harvard CS265 compares YARN's ApplicationMaster model to **Mesos's executor model**: YARN pushes scheduling intelligence into the AM (two-level scheduling), Mesos offers resource offers directly to frameworks (offer-based)

CLO3 Connection

Case Study — LinkedIn YARN Cluster (2013–2016)

LinkedIn's Data Challenge

Why LinkedIn Needed YARN Urgently

LinkedIn (acquired by Microsoft in 2016) ran the world's largest professional network, generating:

- **Profile views, connection requests, job searches**: ≈2 billion page-views/day by 2013
- **People You May Know (PYMK)**: graph algorithm over 300 million member profiles updated daily
- **Skills & Endorsements**: collaborative filtering over 25 billion endorsement events
- **Ad targeting**: real-time bidding backed by offline feature engineering on member activity logs
- **Feed ranking**: news-feed ML models retrained every 4 hours on the latest engagement signals

All of these needed **different frameworks**: graph

The Hadoop 1.x Wall

LinkedIn ran **separate clusters** per workload:

- **Cluster A**: MapReduce for Hive ETL
- **Cluster B**: experimental Spark cluster
- **Cluster C**: Samza stream processing nodes

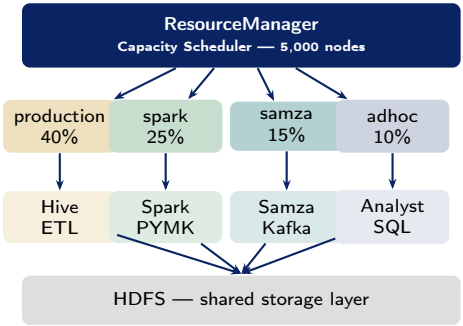
Each cluster was **under-utilised** at different times. Cross-cluster data movement added latency. Managing three separate cluster configurations tripled ops cost.

YARN promised to **collapse all three into one** with fair resource sharing and framework isolation.

LinkedIn's Multi-Framework YARN Architecture

Unified Cluster Design (2013–2016)

- **Single 5,000-node YARN cluster** (Capacity Scheduler with 6 queues)
- **Queue allocation**:
  - `production` (40%): Hive ETL, Pig
  - `spark` (25%): Spark batch + MLlib
  - `samza` (15%): Kafka consumer streams
  - `adhoc` (10%): analyst Hive queries
  - `tez` (7%): interactive Hive-on-Tez
  - `reserve` (3%): overflow / burst
- **Pre-emption enabled**: idle capacity in one queue borrowed by others; reclaimed with 30-second warning
- **Azkaban** (LinkedIn's open-source workflow scheduler) orchestrates cross-queue job pipelines



Scale Numbers & PYMK at LinkedIn

LinkedIn YARN Cluster Scale (2015)

Metric	Value
YARN cluster size	5,000 nodes
Daily data processed	500 TB
Peak containers running	400,000
Daily jobs submitted	30,000+
PYMK Spark job size	300M member graph
PYMK recompute time (YARN)	4 hours (was 18 hr o
Hive queries/day (analysts)	8,000+
Samza Kafka lag (p99)	$<$500 ms
YARN cluster utilisation	78% (vs. 45% on 3 s

People You May Know (PYMK) Pipeline

PYMK is LinkedIn's friend-recommendation feature. Its batch pipeline on YARN:

1. **Ingest** (Samza): Kafka consumer reads member-profile-view events in real time
2. **Feature store** (Hive): nightly ETL joins 1st, 2nd, 3rd-degree connections for all 300M members
3. **Graph model** (Spark GraphX): runs **label propagation** over the social graph to score $\approx$10B candidate pairs
4. **Serve**: top-k recommendations per member written to **Voldemort** (key-value store)

Lessons Learned & Discussion Questions

Key Lessons from LinkedIn's YARN Deployment

1. **Resource consolidation increases utilisation**: merging 3 siloed clusters into one raised average utilisation from 45% to 78% — the same capacity served 73% more workload
2. **Capacity Scheduler enables SLAs**: production queues guaranteed minimum resources; analyst `adhoc` queries cannot starve revenue-generating pipelines
3. **Pre-emption needs tuning**: aggressive pre-emption killed long-running Spark jobs; LinkedIn set 30-second grace periods and added Spark checkpoint-on-preempt hooks
4. **YARN is the OS; frameworks are processes**: the paradigm shift — the cluster is the

Discussion Questions (CLO3, CLO4)

1. LinkedIn's Capacity Scheduler allocates 40% of 5,000 nodes to the `production` queue. If a Spark job in the `spark` queue needs 2,000 containers but only 1,250 are available, describe **three** outcomes under (a) Capacity, (b) Fair, and (c) FIFO schedulers. *[CLO3 — Explain]*
2. LinkedIn's PYMK Spark job processes a 300-million-node social graph. If the job uses 600 executors each with 8 GB RAM, calculate the total memory requested as YARN containers and identify the **dominant resource** if each node has 64 GB RAM and 16 vCPUs. *[CLO4 — Apply]*

Live Exercise

Live Exercise — YARN Capacity Planning Calculator

Task (20 minutes, pairs)

Model LinkedIn's 5,000-node YARN cluster in Python.

Given:

- 5,000 nodes; each: 64 GB RAM, 16 vCPUs
- Capacity Scheduler: production 40%, spark 25%, samza 15%, adhoc 10%, reserve 10%
- Container sizes: Hive mapper: {4 GB, 2 vCPU}; Spark executor: {8 GB, 4 vCPU}; Samza task: {2 GB, 1 vCPU}

Questions:

1. Max concurrent containers per queue?
2. Which resource is the bottleneck (CPU or RAM)?
3. If production needs 10,000 Hive mappers, how

Python Skeleton

```
NODES    = 5_000
RAM_GB   = 64
VCPUS    = 16

queues = {
  "production": 0.40,
  "spark":      0.25,
  "samza":      0.15,
  "adhoc":      0.10,
  "reserve":    0.10,
}
containers = {
  "production": {"ram": 4,  "cpu": 2},
  "spark":      {"ram": 8,  "cpu": 4},
  "samza":      {"ram": 2,  "cpu": 1},
  "adhoc":      {"ram": 4,  "cpu": 2},
}

for q, frac in queues.items():
    nodes_q  = NODES * frac
    ram_q    = nodes_q * RAM_GB
    cpu_q    = nodes_q * VCPUS
    if q in containers:
```

References

Seminal Paper

- Vavilapalli, V. K., et al. (2013). Apache Hadoop YARN: Yet another resource negotiator. *Proc. 4th ACM Symposium on Cloud Computing (SoCC)*.

Textbooks [TW], [SA], [TE]

- **[TW]** White, T. (2015). *Hadoop: The Definitive Guide*, 4th ed. (Ch. 4 — YARN Architecture). O'Reilly.
- **[SA]** Acharya, S., & Chellappan, S. (2015). *Big Data and Analytics* (Ch. 5 — Hadoop Ecosystem Scheduling). Wiley.
- **[TE]** Erl, T., Khattak, W., & Buhler, P. (2016). *Big Data Fundamentals* (Ch. 7 — Cloud-Scale Resource Management). Prentice Hall.

Case Study Sources

- Agarwal, S., et al. (2012). Resumable streams for smart YARN clusters. *LinkedIn Engineering Blog*.
- Garg, A., & Shivastava, Y. (2014). Pooja: Yet Many Keys: Fast computation at LinkedIn

Closing — Week 5, Part 2

Key Takeaways

- YARN's key insight: **separate the global resource scheduler from the per-application execution logic** (ApplicationMaster) — this single decision made Hadoop a general cluster OS instead of a MapReduce runner
- The **Container abstraction** (vCPU + RAM) is the direct ancestor of Kubernetes Pods — cluster resource management has converged on this model
- LinkedIn's deployment proved that **multi-framework consolidation** raises utilisation from 45% to 78% — the business case for YARN was as strong as the engineering case
- **YARN's legacy**: Spark on YARN is still the dominant deployment mode in on-premise

Quote of the Week

"YARN moves Hadoop beyond batch processing, enabling interactive, streaming, and graph workloads on the same cluster."

— Vavilapalli et al. (2013)

Part 1 Preview — Week 6

Apache Spark, RDDs, DataFrames & Lazy Evaluation

- RDD lineage and fault tolerance
- Narrow vs. wide transformations
- Catalyst optimiser and Tungsten engine